

Introduction to C++ Standard Library: Stack, Vector, Queue, and Map

The C++ Standard Library (STL) is a powerful set of template classes and functions that provide a wide range of data structures and algorithms. Here's a brief overview of some of the key components: **Stack**, **Vector**, **Queue**, and **Map**.

1. Stack

- **Definition:** A stack is a container adaptor that follows the Last In First Out (LIFO) principle. This means that the last element added to the stack is the first one to be removed.
- **Underlying Container:** Typically implemented using `deque` or `vector`.
- **Common Operations:**
 - `push()`: Adds an element to the top of the stack.
 - `pop()`: Removes the top element.
 - `top()`: Accesses the top element without removing it.
 - `empty()`: Checks if the stack is empty.
 - `size()`: Returns the number of elements in the stack.

Example Code:

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2#include <stack>
3
4using namespace std;
5
6int main() {
7    stack<int> s;
8    s.push(1);
9    s.push(2);
10   s.push(3);
11
12   cout << "Top element: " << s.top() << endl; // Outputs 3
13   s.pop();
14   cout << "Top element after pop: " << s.top() << endl; // Outputs 2
15   cout << "Stack size: " << s.size() << endl; // Outputs 2
16
17   return 0;
18}
```

2. Vector

- **Definition:** A vector is a dynamic array that can resize itself automatically when elements are added or removed.
- **Key Features:**
 - Provides random access to elements.
 - Supports dynamic resizing.
- **Common Operations:**
 - `push_back()`: Adds an element to the end.
 - `pop_back()`: Removes the last element.

- `size()`: Returns the number of elements.
- `at()`: Accesses an element at a specific index.

Example Code:

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2#include <vector>
3
4using namespace std;
5
6int main() {
7    vector<int> v;
8    v.push_back(10);
9    v.push_back(20);
10   v.push_back(30);
11
12   cout << "Vector elements: ";
13   for (int i = 0; i < v.size(); i++) {
14       cout << v.at(i) << " "; // Outputs 10 20 30
15   }
16   cout << endl;
17
18   return 0;
19}
```

3. Queue

- **Definition:** A queue is a container adaptor that follows the First In First Out (FIFO) principle. Elements are added at the back and removed from the front.
- **Underlying Container:** Typically implemented using `deque` or `list`.
- **Common Operations:**
 - `push()`: Adds an element to the back.
 - `pop()`: Removes the front element.
 - `front()`: Accesses the front element.
 - `back()`: Accesses the last element.
 - `empty()`: Checks if the queue is empty.

Example Code:

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2#include <queue>
3
4using namespace std;
5
6void showQueue(queue<int> q) {
7    while (!q.empty()) {
8        cout << q.front() << " ";
9        q.pop();
10   }
11   cout << endl;
12}
13
14int main() {
```

```
15    queue<int> q;
16    q.push(10);
17    q.push(20);
18    q.push(30);
19
20    cout << "Queue elements: ";
21    showQueue(q); // Outputs 10 20 30
22
23    return 0;
24}
```

4. Map

- **Definition:** A map is an associative container that stores elements in key-value pairs. Each key is unique, and the elements are sorted based on the keys.
- **Common Operations:**
 - `insert()`: Adds a key-value pair.
 - `find()`: Searches for a key.
 - `erase()`: Removes a key-value pair.
 - `size()`: Returns the number of elements.

Example Code:

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2#include <map>
3
4using namespace std;
5
6int main() {
7    map<string, int> m;
8    m["apple"] = 1;
9    m["banana"] = 2;
10   m["cherry"] = 3;
11
12   cout << "Map elements:" << endl;
13   for (const auto& pair : m) {
14       cout << pair.first << ": " << pair.second << endl; // Outputs key-
value pairs
15   }
16
17   return 0;
18}
```

Conclusion

The C++ Standard Library provides a rich set of data structures that can be used to solve various programming problems efficiently. Understanding how to use **Stack**, **Vector**, **Queue**, and **Map** will greatly enhance your ability to write effective C++ programs. Each of these containers has its own strengths and is suited for different types of tasks, so choosing the right one is crucial for optimal performance.

Introduction to RTTI in C++

Run-Time Type Information (RTTI) is a powerful feature in C++ that allows the type of an object to be determined during program execution. This mechanism is particularly useful in scenarios involving polymorphism, where you may need to safely downcast base class pointers to derived class pointers.

Key Concepts of RTTI

- **Polymorphic Types:** RTTI is only available for classes that have at least one virtual function. This is because RTTI relies on the presence of a virtual table (vtable) that stores type information.
- **Operators:** The two primary operators used for RTTI in C++ are:
 - `dynamic_cast`: Used for safe downcasting of pointers and references.
 - `typeid`: Used to obtain type information about an object.

Applications of RTTI

- **Type-safe Downcasting:** Ensures that a base class pointer can be safely cast to a derived class pointer.
- **Debugging:** Helps in identifying the actual type of an object during debugging.
- **Serialization:** Can be used in frameworks to store and retrieve type information.
- **Dynamic Loading:** Useful in applications that support plugins or extensions.

Examples of RTTI in C++

Example 1: Using `dynamic_cast`

This example demonstrates how to use `dynamic_cast` for safe downcasting.

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2#include <memory>
3
4class Base {
5public:
6    virtual ~Base() {} // Virtual destructor
7};
8
9class Derived : public Base {
10public:
11    void show() {
12        std::cout << "Derived class method called." << std::endl;
13    }
14};
15
16int main() {
17    Base* basePtr = new Derived(); // Base class pointer to Derived object
18    Derived* derivedPtr = dynamic_cast<Derived*>(basePtr); // Safe downcast
19
20    if (derivedPtr) {
21        derivedPtr->show(); // Call derived class method
22    } else {
23        std::cout << "Downcasting failed." << std::endl;
24    }
25}
```

```
26     delete basePtr; // Clean up
27     return 0;
28}
```

Output:

VerifyOpen In EditorRunCopy code
1Derived class method called.

Example 2: Using typeid

This example shows how to use `typeid` to get the type information of an object.

```
cpp
VerifyOpen In EditorRunCopy code
1#include <iostream>
2#include <typeinfo>
3
4class Base {
5public:
6    virtual ~Base() {}
7};
8
9class Derived : public Base {};
10
11int main() {
12    Base* basePtr = new Derived();
13    std::cout << "Type of basePtr: " << typeid(*basePtr).name() <<
std::endl; // Get type info
14
15    delete basePtr; // Clean up
16    return 0;
17}
```

Output:

VerifyOpen In EditorRunCopy code
1Type of basePtr: 7Derived

Conclusion

RTTI is a valuable feature in C++ that enhances type safety and flexibility in object-oriented programming. By using `dynamic_cast` and `typeid`, developers can effectively manage polymorphic behavior and ensure safe type conversions. However, it's important to use RTTI judiciously, as excessive reliance on it can lead to code that is harder to maintain.